



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Semestre 2026-2

BASES DE DATOS

Grupo 1

Fernando Arreola Franco

PROYECTO FINAL

FECHA DE ENTREGA: 12 de mayo de 2026

Nombre completo	Número de cuenta
Bojorquez Covarrubias Evans Martin	321203018
Gabriel Hernández Yukioayax Canek	321058526
Martinez Ortiz Eydan	321297178
Parra Bello Carlos Enrique	321135344
Parra Vera Héctor Jesús	321221027
Yañez Barajas Brandon	321237125

CALIFICACIÓN: _____

Índice

1. Introducción	3
2. Plan de trabajo	3
2.1. Descripción general de las actividades	3
2.2. Participación de cada integrante	3
2.3. Tablas del sistema	4
3. Diseño	4
3.1. Modelo entidad-relación y modelo relacional	4
3.2. Diseño del esquema de la base de datos	6
3.3. Tablas del sistema	6
4. Implementación	7
4.1. Secuencias	7
4.1.1. Descripción	7
4.1.2. Implementación	7
4.1.3. Resumen	8
4.2. Creación de tablas	8
4.2.1. Descripción	8
4.2.2. Limpieza previa	8
4.2.3. Tablas de empleados	8
4.2.4. Tablas de productos	10
4.2.5. Tablas de órdenes y facturación	11
4.2.6. Verificación de tablas creadas	12
4.3. Función generar_folio	12
4.3.1. Descripción	12
4.3.2. Implementación	13
4.3.3. Explicación	13
4.3.4. Ejemplos de salida	13
4.3.5. Ventajas de la implementación	14
4.4. Triggers	14
4.4.1. Descripción general	14
4.4.2. Trigger 1: Generación automática de folio	14
4.4.3. Trigger 2: Inserción de detalles de orden	15
4.4.4. Trigger 3: Actualización de detalles de orden	15
4.4.5. Trigger 4: Eliminación de detalles de orden	16
4.4.6. Beneficios de los triggers	16
4.4.7. Funciones de Consulta	17
4.4.8. Función: ordenes_mesero_hoy	17
4.4.9. Función: ventas_por_fechas	18
4.4.10. Pruebas para las Funciones de Consulta	18
4.4.11. Resultados	18
4.5. Vistas	19
4.5.1. Vista platillo más vendido	19
4.5.2. Vista de la factura	20
4.5.3. Vista productos no disponibles	21
4.5.4. Resultados Vistas	21
4.6. Indices	22

5. Presentación	22
5.1. Fase de exportación de la BD origen a txt	22
5.2. Fase de importación de txt a la BD destino	26
5.3. Orquestación y automatización (Trabajos/Jobs)	30
6. Conclusiones	31

1. Introducción

En el presente documento se describe el desarrollo del proyecto final correspondiente a la asignatura de Bases de Datos. En él se analizan los requerimientos del problema propuesto y se presenta la solución desarrollada por nuestro equipo.

El problema consiste en digitalizar la operación de un restaurante mediante una base de datos en PostgreSQL. El objetivo del proyecto es construir una base de datos relacional que centralice la información de empleados, productos y órdenes, automatice los cálculos y validaciones del negocio mediante funciones y triggers, y permita migrar la información registrada durante el día hacia una segunda base de datos de destino.

Para lograrlo se identificaron las entidades que componen el negocio y, a partir de ellas, se definieron las tablas con sus atributos, llaves primarias y llaves foráneas correspondientes. Cada entidad debía satisfacer los requerimientos, por lo que también integramos vistas, triggers, funciones e índices.

2. Plan de trabajo

Esta sección describe las actividades realizadas durante el desarrollo del proyecto, su distribución en el tiempo y la participación de cada integrante del equipo.

2.1. Descripción general de las actividades

El desarrollo del proyecto se dividió en las siguientes fases:

- **Análisis de requerimientos:** Estudio del problema y de los puntos solicitados en el documento del profesor.
- **Diseño:** Elaboración del modelo entidad-relación (MER) y del modelo relacional (MR), y definición de la estrategia de especialización.
- **Implementación de la base de datos:** Secuencias, DDL de las tablas, funciones, triggers, vistas e índices.
- **Funciones de consulta y pruebas:** Desarrollo de las funciones solicitadas y de los scripts de prueba.
- **Migración (ETL):** Exportación de los datos a archivos de texto, importación a la base de datos de destino y orquestación mediante Jobs de Pentaho.
- **Documentación:** Elaboración del presente reporte y de la presentación.

2.2. Participación de cada integrante

Integrante	Responsabilidad principal
Bojorquez Covarrubias Evans Martin	Funciones de consulta y scripts de prueba.
Gabriel Hernández Yukioayax Canek	Orquestación con Pentaho.
Martínez Ortiz Eydan	Vistas (platillo más vendido, factura y productos no disponibles).
Parra Bello Carlos Enrique	Diseño de tablas. Funciones y triggers.
Parra Vera Héctor Jesús	Triggers y consistencia de totales de las órdenes.
Yañez Barajas Brandon	Orquestación con Pentaho: exportación, importación y Jobs.

Tabla 1: Distribución de responsabilidades por integrante

2.3. Tablas del sistema

La base de datos se encuentra compuesta por las siguientes entidades:

Grupo	Tabla	Descripción
Empleados	EMPLEADO	Entidad principal con la información general del personal.
Empleados	COCINERO	Especialización de empleado encargada de almacenar la especialidad culinaria.
Empleados	MESERO	Especialización de empleado encargada de almacenar horarios de atención.
Empleados	ADMINISTRATIVO	Especialización para personal administrativo.
Extras	TELEFONO_EMPLEADO	Almacena los teléfonos asociados a cada empleado.
Extras	DEPENDIENTE	Registra familiares o dependientes del empleado.
Productos	PRODUCTO	Información general de productos ofrecidos por el restaurante.
Productos	PLATILLO	Especialización para alimentos.
Productos	BEBIDA	Especialización para bebidas.
Catálogo	CATEGORIA	Clasificación de productos.
Órdenes	ORDEN	Registro principal de órdenes realizadas por los clientes.
Órdenes	DETALLE_ORDEN	Productos incluidos en cada orden.
Facturación	CLIENTE_FACT	Cientes que solicitan factura.

Tabla 2: Tablas que conforman la base de datos del restaurante

3. Diseño

3.1. Modelo entidad-relación y modelo relacional

A partir del análisis de requerimientos se elaboró el modelo entidad-relación (MER) y, posteriormente, el modelo relacional (MR) que dio origen al esquema implementado.

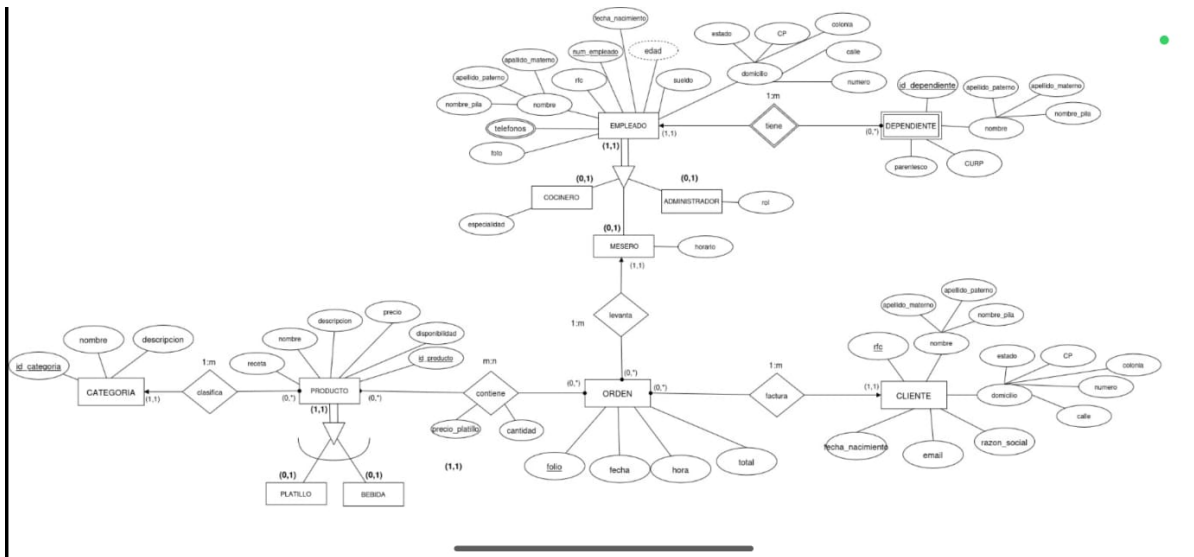


Figura 1: Modelo entidad-relación (MER) del sistema.

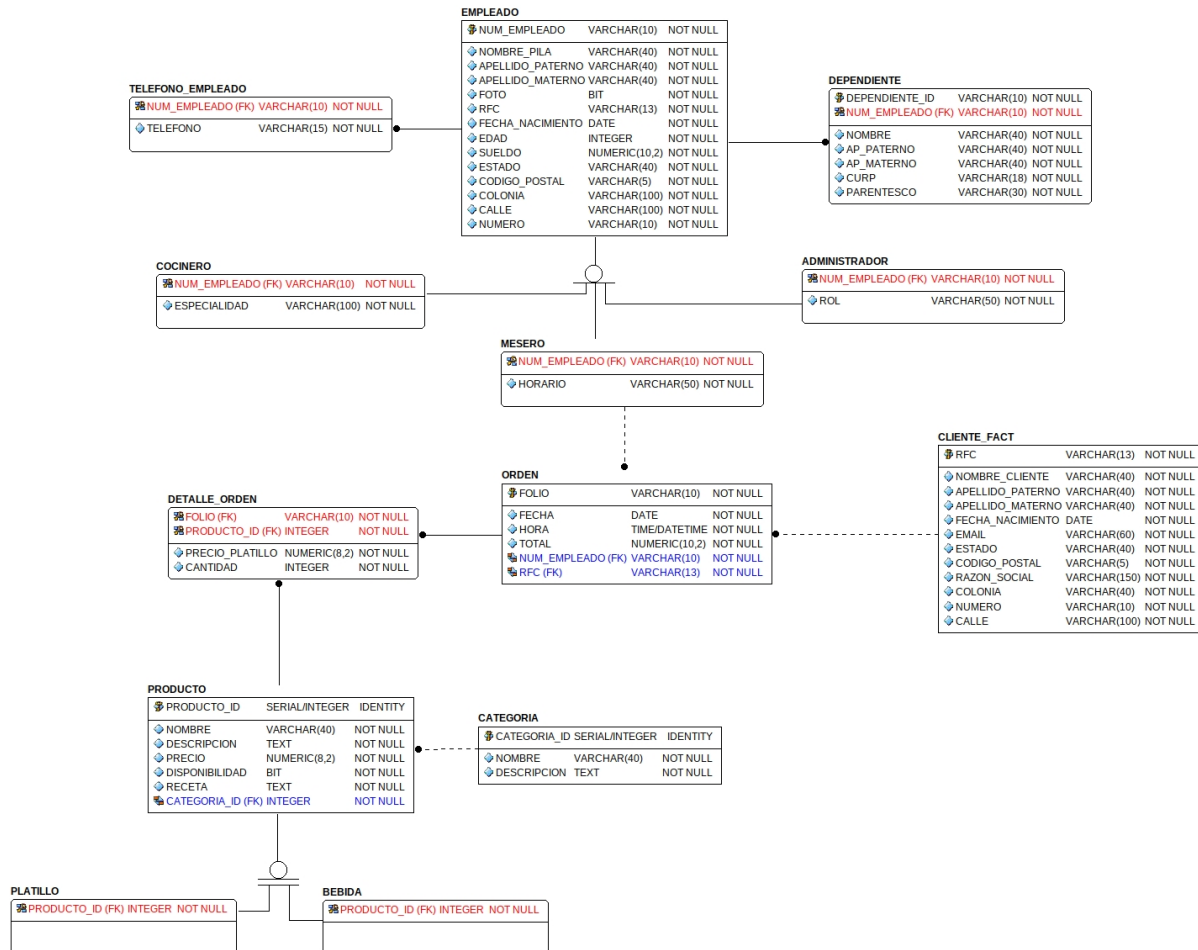


Figura 2: Modelo relacional (MR) del sistema.

3.2. Diseño del esquema de la base de datos

Para el modelado de la base de datos se utilizó la estrategia conocida como *tabla por subclase* (*table per subclass*), la cual permite representar correctamente las especializaciones presentes en las entidades principales del sistema.

En particular, esta estrategia se empleó para las entidades EMPLEADO y PRODUCTO, permitiendo que cada subtipo almacene sus atributos específicos en tablas independientes mientras conserva la integridad referencial con la entidad padre.

Las principales ventajas de esta estrategia son:

- Cada subtipo posee una tabla propia vinculada mediante llaves foráneas a la entidad principal.
- Reduce la redundancia de información al evitar la duplicación de atributos comunes.
- Facilita el mantenimiento y la escalabilidad del esquema de datos.
- Permite que un mismo empleado pueda desempeñar varios puestos a la vez, ya que puede existir simultáneamente en más de una tabla de subtipo (por ejemplo, en COCINERO y MESERO), tal como lo exige el requerimiento.

3.3. Tablas del sistema

La base de datos se encuentra compuesta por las siguientes entidades:

Grupo	Tabla	Descripción
Empleados	EMPLEADO	Entidad principal con la información general del personal.
Empleados	COCINERO	Especialización de empleado encargada de almacenar la especialidad culinaria.
Empleados	MESERO	Especialización de empleado encargada de almacenar horarios de atención.
Empleados	ADMINISTRATIVO	Especialización para personal administrativo.
Extras	TELEFONO_EMPLEADO	Almacena los teléfonos asociados a cada empleado.
Extras	DEPENDIENTE	Registra familiares o dependientes del empleado.
Productos	PRODUCTO	Información general de productos ofrecidos por el restaurante.
Productos	PLATILLO	Especialización para alimentos.
Productos	BEBIDA	Especialización para bebidas.
Catálogo	CATEGORIA	Clasificación de productos.
Órdenes	ORDEN	Registro principal de órdenes realizadas por los clientes.
Órdenes	DETALLE_ORDEN	Productos incluidos en cada orden.
Facturación	CLIENTE_FACT	Cientes que solicitan factura.

Tabla 3: Tablas que conforman la base de datos del restaurante

4. Implementación

4.1. Secuencias

4.1.1. Descripción

Para automatizar la generación de identificadores únicos dentro de la base de datos se implementaron diversas secuencias en PostgreSQL. Estas secuencias permiten generar valores autoincrementales que posteriormente son utilizados como llaves primarias o identificadores internos de distintas entidades del sistema. Las secuencias se ejecutan antes de la creación de las tablas, ya que algunas de ellas son referenciadas mediante la función `nextval()` para asignar automáticamente nuevos identificadores durante las operaciones de inserción.

4.1.2. Implementación

La secuencia encargada de generar los folios de las órdenes se define de la siguiente manera:

```
1 DROP SEQUENCE IF EXISTS folio_seq;
2
3 CREATE SEQUENCE folio_seq
4     START WITH 1
5     INCREMENT BY 1
6     MINVALUE 1
7     NO MAXVALUE
8     NO CYCLE;
```

Donde:

- `START WITH 1` indica que la secuencia comenzará en el valor 1.
- `INCREMENT BY 1` incrementa el contador una unidad por cada llamada.
- `MINVALUE 1` impide que se generen valores menores a uno.
- `NO MAXVALUE` elimina una cota superior para la secuencia.
- `NO CYCLE` evita que la secuencia vuelva a comenzar cuando alcance valores elevados.

De forma similar se definieron secuencias para otras entidades del sistema:

```
1 DROP SEQUENCE IF EXISTS dependiente_id_seq;
2 CREATE SEQUENCE dependiente_id_seq
3     START WITH 1
4     INCREMENT BY 1
5     MINVALUE 1
6     NO MAXVALUE
7     NO CYCLE;
8
9 DROP SEQUENCE IF EXISTS categoria_id_seq;
10 CREATE SEQUENCE categoria_id_seq
11     START WITH 1
12     INCREMENT BY 1
13     MINVALUE 1
14     NO MAXVALUE
15     NO CYCLE;
16
17 DROP SEQUENCE IF EXISTS producto_id_seq;
18 CREATE SEQUENCE producto_id_seq
```



```

19  START WITH 1
20  INCREMENT BY 1
21  MINVALUE 1
22  NO MAXVALUE
23  NO CYCLE;

```

Estas secuencias son utilizadas para generar automáticamente los identificadores de las tablas **DEPENDIENTE**, **CATEGORIA** y **PRODUCTO**, garantizando unicidad y evitando conflictos durante la inserción de registros.

4.1.3. Resumen

Secuencia	Tabla	Propósito
folio_seq	ORDEN	Generación de folios de órdenes
dependiente_id_seq	DEPENDIENTE	Identificador de dependientes
categoria_id_seq	CATEGORIA	Identificador de categorías
producto_id_seq	PRODUCTO	Identificador de productos

Tabla 4: Secuencias utilizadas en la base de datos

4.2. Creación de tablas

4.2.1. Descripción

El archivo `01_ddl_creacion_tablas.sql` contiene la creación de las tablas principales del sistema de restaurante. En este script se definen las entidades relacionadas con empleados, productos, órdenes, clientes con facturación y detalles de las órdenes.

Las tablas se crean respetando el orden de dependencia entre ellas, ya que algunas contienen llaves foráneas que apuntan hacia otras tablas previamente definidas.

4.2.2. Limpieza previa

Antes de crear las tablas, el script elimina las tablas existentes para evitar conflictos al volver a ejecutar el archivo.

```

1  DROP TABLE IF EXISTS DETALLE_ORDEN CASCADE;
2  DROP TABLE IF EXISTS ORDEN CASCADE;
3  DROP TABLE IF EXISTS BEBIDA CASCADE;
4  DROP TABLE IF EXISTS PLATILLO CASCADE;
5  DROP TABLE IF EXISTS PRODUCTO CASCADE;
6  DROP TABLE IF EXISTS CATEGORIA CASCADE;
7  DROP TABLE IF EXISTS CLIENTE_FACT CASCADE;
8  DROP TABLE IF EXISTS ADMINISTRATIVO CASCADE;
9  DROP TABLE IF EXISTS COCINERO CASCADE;
10 DROP TABLE IF EXISTS MESERO CASCADE;
11 DROP TABLE IF EXISTS DEPENDIENTE CASCADE;
12 DROP TABLE IF EXISTS TELEFONO_EMPLEADO CASCADE;
13 DROP TABLE IF EXISTS EMPLEADO CASCADE;

```

La instrucción `IF EXISTS` evita errores si la tabla no existe. La opción `CASCADE` permite eliminar también las restricciones relacionadas con esa tabla.

4.2.3. Tablas de empleados

La tabla **EMPLEADO** funciona como entidad principal del personal del restaurante. Almacena datos generales como nombre, RFC, fecha de nacimiento, sueldo y dirección.

```

1 CREATE TABLE EMPLEADO (
2     num_empleado VARCHAR(10) NOT NULL,
3     nombre_pila VARCHAR(40) NOT NULL,
4     apellido_paterno VARCHAR(40) NOT NULL,
5     apellido_materno VARCHAR(40) NOT NULL,
6     rfc VARCHAR(13) NOT NULL,
7     fecha_nacimiento DATE NOT NULL,
8     edad INTEGER NOT NULL,
9     sueldo NUMERIC(10,2) NOT NULL CHECK (sueldo > 0),
10    foto BYTEA,
11    estado VARCHAR(40) NOT NULL,
12    codigo_postal VARCHAR(5) NOT NULL,
13    colonia VARCHAR(100) NOT NULL,
14    calle VARCHAR(100) NOT NULL,
15    numero VARCHAR(10) NOT NULL,
16    CONSTRAINT pk_empleado PRIMARY KEY (num_empleado),
17    CONSTRAINT uq_empleado_rfc UNIQUE (rfc)
18 );

```

La llave primaria es num_empleado. Además, el campo rfc se define como único para evitar registrar dos empleados con el mismo RFC.

La tabla TELEFONO_EMPLEADO representa un atributo multivaluado, ya que un empleado puede tener más de un número telefónico.

```

1 CREATE TABLE TELEFONO_EMPLEADO (
2     num_empleado VARCHAR(10) NOT NULL,
3     telefono VARCHAR(15) NOT NULL,
4     CONSTRAINT pk_telefono PRIMARY KEY (num_empleado, telefono),
5     CONSTRAINT fk_telefono_empleado FOREIGN KEY (num_empleado)
6         REFERENCES EMPLEADO (num_empleado)
7         ON DELETE CASCADE
8         ON UPDATE CASCADE
9 );

```

La llave primaria compuesta (num_empleado, telefono) evita que un mismo teléfono se repita para el mismo empleado.

La tabla DEPENDIENTE almacena los familiares o dependientes asociados a cada empleado.

```

1 CREATE TABLE DEPENDIENTE (
2     dependiente_id INTEGER NOT NULL DEFAULT nextval('dependiente_id_seq'),
3     num_empleado VARCHAR(10) NOT NULL,
4     nombre VARCHAR(40) NOT NULL,
5     ap_paterno VARCHAR(40) NOT NULL,
6     ap_materno VARCHAR(40) NOT NULL,
7     curp VARCHAR(18) NOT NULL,
8     parentesco VARCHAR(30) NOT NULL,
9     CONSTRAINT pk_dependiente PRIMARY KEY (dependiente_id),
10    CONSTRAINT uq_dependiente_curp UNIQUE (curp),
11    CONSTRAINT fk_dependiente_empleado FOREIGN KEY (num_empleado)
12        REFERENCES EMPLEADO (num_empleado)
13        ON DELETE CASCADE
14        ON UPDATE CASCADE
15 );

```

El identificador dependiente_id se genera automáticamente mediante una secuencia. La restricción UNIQUE sobre curp impide duplicar dependientes.

Las tablas COCINERO, MESERO y ADMINISTRATIVO representan subtipos de EMPLEADO.

```

1 CREATE TABLE COCINERO (
2     num_empleado VARCHAR(10) NOT NULL,
3     especialidad VARCHAR(100) NOT NULL,
4     CONSTRAINT pk_cocinero PRIMARY KEY (num_empleado),
5     CONSTRAINT fk_cocinero_empleado FOREIGN KEY (num_empleado)
6         REFERENCES EMPLEADO (num_empleado)
7         ON DELETE CASCADE
8         ON UPDATE CASCADE
9 );
10
11 CREATE TABLE MESERO (
12     num_empleado VARCHAR(10) NOT NULL,
13     horario VARCHAR(50) NOT NULL,
14     CONSTRAINT pk_mesero PRIMARY KEY (num_empleado),
15     CONSTRAINT fk_mesero_empleado FOREIGN KEY (num_empleado)
16         REFERENCES EMPLEADO (num_empleado)
17         ON DELETE CASCADE
18         ON UPDATE CASCADE
19 );
20
21 CREATE TABLE ADMINISTRATIVO (
22     num_empleado VARCHAR(10) NOT NULL,
23     rol VARCHAR(50) NOT NULL,
24     CONSTRAINT pk_administrativo PRIMARY KEY (num_empleado),
25     CONSTRAINT fk_administrativo_empleado FOREIGN KEY (num_empleado)
26         REFERENCES EMPLEADO (num_empleado)
27         ON DELETE CASCADE
28         ON UPDATE CASCADE
29 );

```

Cada subtipo utiliza como llave primaria el mismo num_empleado, manteniendo la relación directa con la tabla padre.

4.2.4. Tablas de productos

La tabla CATEGORIA permite clasificar los productos del restaurante.

```

1 CREATE TABLE CATEGORIA (
2     categoria_id INTEGER NOT NULL DEFAULT nextval('categoria_id_seq'),
3     nombre VARCHAR(40) NOT NULL,
4     descripcion TEXT NOT NULL,
5     CONSTRAINT pk_categoria PRIMARY KEY (categoria_id)
6 );

```

La tabla PRODUCTO almacena la información general de los productos ofrecidos, como nombre, descripción, precio, disponibilidad, receta y categoría.

```

1 CREATE TABLE PRODUCTO (
2     producto_id INTEGER NOT NULL DEFAULT nextval('producto_id_seq'),
3     nombre VARCHAR(40) NOT NULL,
4     descripcion TEXT NOT NULL,
5     precio NUMERIC(8,2) NOT NULL CHECK (precio > 0),
6     disponibilidad BOOLEAN NOT NULL DEFAULT TRUE,
7     receta TEXT NOT NULL,
8     categoria_id INTEGER NOT NULL,
9     tipo VARCHAR(10) NOT NULL CHECK (tipo IN ('platillo', 'bebida')),

```

```

10     CONSTRAINT pk_producto PRIMARY KEY (producto_id),
11     CONSTRAINT fk_producto_categoria FOREIGN KEY (categoria_id)
12         REFERENCES CATEGORIA (categoria_id)
13         ON DELETE RESTRICT
14         ON UPDATE CASCADE
15 );

```

El atributo `tipo` funciona como discriminador, indicando si el producto corresponde a un platillo o a una bebida. El campo `disponibilidad` permite controlar si el producto puede ser vendido.

Las tablas `PLATILLO` y `BEBIDA` son subtipos de `PRODUCTO`.

```

1 CREATE TABLE PLATILLO (
2     producto_id INTEGER NOT NULL,
3     CONSTRAINT pk_platillo PRIMARY KEY (producto_id),
4     CONSTRAINT fk_platillo_producto FOREIGN KEY (producto_id)
5         REFERENCES PRODUCTO (producto_id)
6         ON DELETE CASCADE
7         ON UPDATE CASCADE
8 );
9
10 CREATE TABLE BEBIDA (
11     producto_id INTEGER NOT NULL,
12     CONSTRAINT pk_bebida PRIMARY KEY (producto_id),
13     CONSTRAINT fk_bebida_producto FOREIGN KEY (producto_id)
14         REFERENCES PRODUCTO (producto_id)
15         ON DELETE CASCADE
16         ON UPDATE CASCADE
17 );

```

4.2.5. Tablas de órdenes y facturación

La tabla `CLIENTE_FACT` registra únicamente a los clientes que solicitan factura.

```

1 CREATE TABLE CLIENTE_FACT (
2     rfc VARCHAR(13) NOT NULL,
3     nombre_cliente VARCHAR(40) NOT NULL,
4     apellido_paterno VARCHAR(40) NOT NULL,
5     apellido_materno VARCHAR(40) NOT NULL,
6     fecha_nacimiento DATE NOT NULL,
7     email VARCHAR(60) NOT NULL,
8     estado VARCHAR(40) NOT NULL,
9     codigo_postal VARCHAR(5) NOT NULL,
10    colonia VARCHAR(40) NOT NULL,
11    numero VARCHAR(10) NOT NULL,
12    calle VARCHAR(100) NOT NULL,
13    razon_social VARCHAR(150) NOT NULL,
14    CONSTRAINT pk_cliente_fact PRIMARY KEY (rfc)
15 );

```

La tabla `ORDEN` representa una orden atendida por un mesero. Contiene el folio, fecha, hora, total, empleado responsable y, opcionalmente, el RFC del cliente facturado.

```

1 CREATE TABLE ORDEN (
2     folio VARCHAR(10) NOT NULL,
3     fecha DATE NOT NULL DEFAULT CURRENT_DATE,
4     hora TIME NOT NULL DEFAULT CURRENT_TIME,
5     total NUMERIC(10,2) NOT NULL DEFAULT 0.00,

```

```

6      num_empleado VARCHAR(10) NOT NULL ,
7      rfc VARCHAR(13),
8      CONSTRAINT pk_orden PRIMARY KEY (folio),
9      CONSTRAINT fk_orden_mesero FOREIGN KEY (num_empleado)
10         REFERENCES MESERO (num_empleado)
11         ON DELETE RESTRICT
12         ON UPDATE CASCADE,
13      CONSTRAINT fk_orden_cliente FOREIGN KEY (rfc)
14         REFERENCES CLIENTE_FACT (rfc)
15         ON DELETE SET NULL
16         ON UPDATE CASCADE
17 );

```

La llave foránea `fk_orden_mesero` asegura que toda orden esté asociada a un mesero existente. En cambio, la llave foránea hacia `CLIENTE_FACT` permite valores nulos, ya que no todas las órdenes requieren factura. Finalmente, la tabla `DETALLE_ORDEN` representa la relación entre una orden y los productos incluidos en ella.

```

1 CREATE TABLE DETALLE_ORDEN (
2     folio VARCHAR(10) NOT NULL ,
3     producto_id INTEGER NOT NULL ,
4     cantidad INTEGER NOT NULL CHECK (cantidad > 0),
5     precio_platillo NUMERIC(8,2) NOT NULL CHECK (precio_platillo > 0),
6     CONSTRAINT pk_detalle_orden PRIMARY KEY (folio, producto_id),
7     CONSTRAINT fk_detalle_orden FOREIGN KEY (folio)
8         REFERENCES ORDEN (folio)
9         ON DELETE CASCADE
10        ON UPDATE CASCADE,
11     CONSTRAINT fk_detalle_producto FOREIGN KEY (producto_id)
12        REFERENCES PRODUCTO (producto_id)
13        ON DELETE RESTRICT
14        ON UPDATE CASCADE
15 );

```

La llave primaria compuesta (`folio`, `producto_id`) evita que un mismo producto se registre más de una vez dentro de la misma orden. Además, la restricción `CHECK (cantidad > 0)` garantiza que solo se registren cantidades válidas.

4.2.6. Verificación de tablas creadas

Al final del script se ejecuta una consulta al catálogo de PostgreSQL para comprobar que las tablas fueron creadas correctamente.

```

1 SELECT table_name
2 FROM information_schema.tables
3 WHERE table_schema = 'public'
4 ORDER BY table_name;

```

Esta consulta muestra todas las tablas pertenecientes al esquema `public`, permitiendo verificar que el proceso de creación se haya completado exitosamente.

4.3. Función generar_folio

4.3.1. Descripción

Con el objetivo de automatizar la generación de identificadores para las órdenes del restaurante, se desarrolló la función `generar_folio()`. Esta función consume valores de la secuencia `folio_seq` y construye

folios con el formato ORD-XXX, donde la parte numérica corresponde al valor generado por la secuencia. La función es utilizada posteriormente por un trigger encargado de asignar automáticamente el folio a cada nueva orden registrada en el sistema.

4.3.2. Implementación

```

1 CREATE OR REPLACE FUNCTION generar_folio()
2 RETURNS VARCHAR(10) AS $$
3 DECLARE
4     v_numero INTEGER;
5     v_folio VARCHAR(10);
6 BEGIN
7     v_numero := nextval('folio_seq');
8
9     v_folio := 'ORD-' || LPAD(v_numero::TEXT, 3, '0');
10
11     RETURN v_folio;
12 END;
13 $$ LANGUAGE plpgsql;

```

4.3.3. Explicación

La función utiliza dos variables locales:

- **v_numero**: almacena temporalmente el valor obtenido de la secuencia.
- **v_folio**: almacena el folio ya formateado.

El valor de la secuencia se obtiene mediante:

```

1 v_numero := nextval('folio_seq');

```

Cada llamada a `nextval()` incrementa automáticamente la secuencia y garantiza que el número generado no se repita.

Posteriormente se construye el folio mediante:

```

1 v_folio := 'ORD-' || LPAD(v_numero::TEXT, 3, '0');

```

La función `LPAD()` agrega ceros a la izquierda cuando el número posee menos de tres dígitos, permitiendo mantener un formato uniforme para todos los folios.

Finalmente, el resultado es retornado al proceso que invocó la función.

4.3.4. Ejemplos de salida

Valor de secuencia	Folio generado
1	ORD-001
50	ORD-050
99	ORD-099
100	ORD-100
1000	ORD-1000

Tabla 5: Ejemplos de folios generados

4.3.5. Ventajas de la implementación

- Garantiza unicidad en los folios generados.
- Evita que el usuario capture manualmente identificadores.
- Facilita la trazabilidad de las órdenes.
- Mantiene un formato homogéneo para todos los registros.
- Aprovecha las secuencias de PostgreSQL para evitar duplicados.

4.4. Triggers

4.4.1. Descripción general

Los triggers implementados en el sistema permiten automatizar diversos procesos relacionados con la gestión de órdenes dentro del restaurante. Su principal objetivo es mantener la integridad de los datos, evitar errores de captura y asegurar que los cálculos realizados sobre las órdenes sean consistentes.

En total se implementaron cuatro triggers asociados a las tablas `ORDEN` y `DETALLE_ORDEN`. Estos triggers permiten generar automáticamente folios para las órdenes, validar productos antes de agregarlos a una orden, recalcular subtotales cuando se modifica una cantidad y actualizar el total de una orden cuando se elimina algún producto.

#	Trigger	Tabla	Evento
1	trg_generar_folio	ORDEN	BEFORE INSERT
2	trg_insertar_detalle	DETALLE_ORDEN	BEFORE INSERT
3	trg_actualizar_detalle	DETALLE_ORDEN	BEFORE UPDATE
4	trg_eliminar_detalle	DETALLE_ORDEN	AFTER DELETE

Tabla 6: Triggers implementados en el sistema

4.4.2. Trigger 1: Generación automática de folio

Este trigger se ejecuta antes de insertar una nueva orden. Su función consiste en generar automáticamente el folio utilizando la función `generar_folio()`, evitando que el usuario capture manualmente dicho identificador.

```

1 CREATE OR REPLACE FUNCTION fn_generar_folio_orden()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     NEW.folio := generar_folio();
5     RETURN NEW;
6 END;
7 $$ LANGUAGE plpgsql;
8
9 CREATE TRIGGER trg_generar_folio
10 BEFORE INSERT ON ORDEN
11 FOR EACH ROW
12 EXECUTE FUNCTION fn_generar_folio_orden();

```

Gracias a este mecanismo, cada orden recibe automáticamente un folio único con formato `ORD-001`, `ORD-002`, `ORD-003`, etc.

4.4.3. Trigger 2: Inserción de detalles de orden

Este trigger se ejecuta antes de insertar un nuevo registro en la tabla DETALLE_ORDEN. Su propósito es validar que el producto exista, verificar que se encuentre disponible para venta, calcular automáticamente el subtotal correspondiente y actualizar el total acumulado de la orden.

```

1 CREATE OR REPLACE FUNCTION fn_insertar_detalle_orden()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     v_disponible BOOLEAN;
5     v_precio NUMERIC(8,2);
6     v_nombre VARCHAR(40);
7 BEGIN
8     SELECT disponibilidad, precio, nombre
9     INTO v_disponible, v_precio, v_nombre
10    FROM PRODUCTO
11   WHERE producto_id = NEW.producto_id;
12
13     IF NOT FOUND THEN
14         RAISE EXCEPTION 'El producto con ID % no existe.',
15                          NEW.producto_id;
16     END IF;
17
18     IF v_disponible = FALSE THEN
19         RAISE EXCEPTION 'El producto "%" no esta disponible.',
20                          v_nombre;
21     END IF;
22
23     NEW.precio_platillo := NEW.cantidad * v_precio;
24
25     UPDATE ORDEN
26     SET total = (
27         SELECT COALESCE(SUM(precio_platillo),0)
28         + NEW.precio_platillo
29         FROM DETALLE_ORDEN
30         WHERE folio = NEW.folio
31     )
32     WHERE folio = NEW.folio;
33
34     RETURN NEW;
35 END;
36 $$ LANGUAGE plpgsql;

```

Este trigger garantiza que solamente se registren productos válidos dentro de una orden y que el cálculo del subtotal se realice automáticamente utilizando el precio almacenado en el catálogo de productos.

4.4.4. Trigger 3: Actualización de detalles de orden

Cuando se modifica la cantidad de un producto dentro de una orden, este trigger recalcula el subtotal correspondiente y actualiza el total general de la orden para reflejar el nuevo valor.

```

1 CREATE OR REPLACE FUNCTION fn_actualizar_detalle_orden()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     v_precio NUMERIC(8,2);
5 BEGIN
6     SELECT precio

```



```

7      INTO v_precio
8      FROM PRODUCTO
9      WHERE producto_id = NEW.producto_id;
10
11     NEW.precio_platillo :=
12         NEW.cantidad * v_precio;
13
14     UPDATE ORDEN
15     SET total = (
16         SELECT COALESCE(SUM(
17             CASE
18                 WHEN folio = NEW.folio
19                     AND producto_id = NEW.producto_id
20                     THEN NEW.precio_platillo
21                     ELSE precio_platillo
22             END
23         ),0)
24         FROM DETALLE_ORDEN
25         WHERE folio = NEW.folio
26     )
27     WHERE folio = NEW.folio;
28
29     RETURN NEW;
30 END;
31 $$ LANGUAGE plpgsql;

```

La utilización de este trigger permite que cualquier modificación realizada en los detalles de una orden se refleje inmediatamente en el total almacenado, evitando inconsistencias entre los datos.

4.4.5. Trigger 4: Eliminación de detalles de orden

Este trigger se ejecuta después de eliminar un producto de una orden. Su función principal consiste en recalculer el total considerando únicamente los productos que permanecen registrados dentro de la orden.

```

1  CREATE OR REPLACE FUNCTION fn_eliminar_detalle_orden()
2  RETURNS TRIGGER AS $$
3  BEGIN
4      UPDATE ORDEN
5      SET total = (
6          SELECT COALESCE(SUM(precio_platillo),0)
7          FROM DETALLE_ORDEN
8          WHERE folio = OLD.folio
9              AND producto_id != OLD.producto_id
10      )
11      WHERE folio = OLD.folio;
12
13      RETURN OLD;
14 END;
15 $$ LANGUAGE plpgsql;

```

A diferencia de los triggers anteriores, este utiliza el evento **AFTER DELETE**, ya que requiere que el registro haya sido eliminado previamente para poder recalculer correctamente el total de la orden.

4.4.6. Beneficios de los triggers

La implementación de estos triggers aporta diversas ventajas al sistema:

- Generación automática de folios para las órdenes.
- Validación de productos antes de registrarlos.
- Cálculo automático de subtotales.
- Actualización automática de totales.
- Reducción de errores de captura.
- Mayor integridad y consistencia de la información almacenada.
- Automatización de procesos repetitivos dentro del sistema.

4.4.7. Funciones de Consulta

Aquí se ve la implementación de las funciones necesarias para consultar la información solicitada en los requerimientos del proyecto. Se programaron en total dos funciones principales:

4.4.8. Función: ordenes_mesero_hoy

Esta función es para calcular las ventas diarias de un mesero, recibe el número de un empleado como parámetro y calcula cuántas órdenes atendió en el día de hoy, así como el dinero total que se cobró. Primero se hizo un SELECT EXISTS en la tabla MESERO para confirmar que el empleado realmente tenga ese puesto.

```

1 CREATE OR REPLACE FUNCTION ordenes_mesero_hoy(p_num_empleado VARCHAR(10))
2 RETURNS TABLE (cantidad_ordenes BIGINT, total_cobrado NUMERIC) AS $$
3 DECLARE
4     v_es_mesero BOOLEAN;
5 BEGIN
6     SELECT EXISTS(SELECT 1 FROM mesero WHERE num_empleado = p_num_empleado)
7     INTO v_es_mesero;

```

Si el empleado no existe o no es un mesero, utilizamos RAISE EXCEPTION para detener la consulta y mostrar un mensaje de error, el mensaje es tal como lo pide el requerimiento.

```

1     IF NOT v_es_mesero THEN
2         RAISE EXCEPTION 'Error: El empleado con número % no es un mesero
3         válido.', p_num_empleado;
4     END IF;

```

Después, para evitar que la función devuelva valores nulos (NULL) en caso de que el mesero no haya tenido ventas en todo el día, usamos COALESCE para que la suma total devuelva un 0 limpio y no un espacio vacío.

```

1     RETURN QUERY
2     SELECT
3         COUNT(folio)::BIGINT,
4         COALESCE(SUM(total), 0)::NUMERIC
5     FROM orden
6     WHERE num_empleado = p_num_empleado
7         AND fecha = CURRENT_DATE;
8 END;
9 $$ LANGUAGE plpgsql;

```

4.4.9. Función: ventas_por_fechas

Esta función nos ayuda a obtener el total de ventas y el dinero ingresado en un periodo de tiempo. Si el usuario no envía una fecha de fin, el sistema asume automáticamente que solo queremos consultar las ventas de ese día en específico.

Para esto, usamos un IF que comprueba si `p_fecha_fin` está vacío NULL. Si es así, le asignamos la misma fecha de inicio, permitiendo que la misma función sirva tanto para reportes diarios como para rangos de fechas más grandes.

```

1 CREATE OR REPLACE FUNCTION ventas_por_fechas(p_fecha_inicio DATE, p_fecha_fin
  DATE DEFAULT NULL)
2 RETURNS TABLE (total_ventas BIGINT, monto_total NUMERIC) AS $$
3 BEGIN
4     IF p_fecha_fin IS NULL THEN
5         p_fecha_fin := p_fecha_inicio;
6     END IF;

```

Por último, regresa una consulta filtrada por el rango de fechas con `RETURN QUERY` para devolver el resultado directamente como una tabla. La operación realiza un conteo de órdenes y una sumatoria el monto total mediante `SUM(total)` y le aplicamos la función `COALESCE` para garantizar que, en caso de no existir registros en el periodo, la función retorne un valor numérico de cero en vez de NULL.

```

1     RETURN QUERY
2     SELECT
3         COUNT(folio)::BIGINT,
4         COALESCE(SUM(total), 0.00)::NUMERIC
5     FROM orden
6     WHERE fecha >= p_fecha_inicio
7         AND fecha <= p_fecha_fin;
8 END;
9 $$ LANGUAGE plpgsql;

```

4.4.10. Pruebas para las Funciones de Consulta

Corresponde al script `test_funciones.sql`, fue diseñado para ejecutarse de forma independiente. Al inicio del archivo, hacemos una limpieza de las tablas y reiniciamos las secuencias para evitar conflictos con datos anteriores, se ocupa un `DELETE` por lo que tenemos que tener mucho cuidado al ejecutar este script. Después, insertamos datos de prueba básicos como una categoría, un empleado que se le asigna rol de mesero, par de productos y una orden. Teniendo todo listo, podemos evaluar las funciones.

4.4.11. Resultados

Esta prueba es para validar que las funciones hagan los cálculos correctos y que los candados de seguridad funcionen bien. Para evaluar el caso del mesero inválido sin que el script marcara un error fatal y detuviera la ejecución de las demás pruebas, metimos la consulta dentro de un bloque anónimo (`DO $$ ... EXCEPTION`). Esto funciona de manera similar a un `try-catch` en programación orientada a objetos: el bloque atrapa la excepción lanzada por la función para que el resto del archivo SQL pueda seguir corriendo con normalidad.

Prueba	Operación	Resultado esperado
1	<code>ordenes_mesero_hoy('EMP-999')</code>	El bloque <code>EXCEPTION</code> captura correctamente el error lanzado porque el empleado no es mesero.
2	<code>ordenes_mesero_hoy('EMP-001')</code>	Devuelve una tabla con 1 orden procesada y \$195.00 de total cobrado.
3	<code>ventas_por_fechas(CURRENT_DATE)</code>	Devuelve el total de ventas y el dinero ingresado del día de hoy.
4	<code>ventas_por_fechas('2026-05-01',...)</code>	Devuelve el acumulado de ventas exacto de todo el mes de mayo.

Tabla 7: Resultados de los casos de prueba para las funciones de consulta.

4.5. Vistas

Unos de los requerimientos planteados era hacer el uso de vistas para así mostrar consultas, segun lo que nos pida que se deba mostrar. Las vistas a resolver fueron: mostrar, platillo más vendido, mostrar una consulta tipo factura y una vista para productos no disponibles.

4.5.1. Vista platillo más vendido

En este apartado se requiere mostrar los detalles del platillo más vendido. Para esto comnzamos creando nuestra vista y le ponemos el nombre de `vista_platillo_mas_vendido`:

```
1 CREATE VIEW vista_platillo_mas_vendido AS
```

Despues comnezamos a describir cuales son los atributos que queremos que se tengan en cuenta para mostrar en la vista, donde es importante recalcar que se necesita la informacion de dos tablas, una de la tabla(entidad) `PRODUCTO` y otra de la tabla(relación) `DETALLE_ORDEN`; donde esta última es necesaria para así poder tener la información de la cantidad del producto y asi calcular con ayuda de la función `SUM()` la cantidad total de platillos vendidos.

```
1 SELECT
2     PRODUCTO.producto_id,
3     PRODUCTO.nombre,
4     PRODUCTO.descripcion,
5     PRODUCTO.precio,
6     SUM(DETALLE_ORDEN.cantidad) AS cant_total
```

Ahora pasamos a hacer un `JOIN` entre las tablas `PRODUCTO` y `DETALLE_ORDEN`, esto para complementar la información faltante en la tabla `PRODUCTO`, y así poder mostrarla en una sola tabla. Donde tambien le diremos cual es la relación entre ambas tablas usando el atributo que tengan en común.

```
1 FROM PRODUCTO
2 JOIN DETALLE_ORDEN ON PRODUCTO.producto_id = DETALLE_ORDEN.producto_id
```

Como el requerimiento solo nos pide platillos tenemos que especificarle a la base de datos que solo tenga en cuenta los platillos e ignore las bebidas, para esto usamos un `where` que vaya hacia el atributo `tipo`, que es el atributo donde se almacena si es platillo o bebida (gracias al `CHECK`)

```
1 WHERE PRODUCTO.tipo = 'platillo'
```

Como usamos la función `SUM()` es vital poner un `GROUP BY` para así decirle a la base de datos bajo que atributo queremos encapsularlo, que en este caso será el `producto_id`. Despues ordenamos la tabla bajo la cantidad total calculada en la suma gracias a un `ORDER BY` de manera descendente para así saber que

platillo es el más vendido (este platillo estara a la cabeza de la lista) y por último para mostrar solo el platillo más vendido usamos un LIMIT 1 para solo limitar la tabla a su primera fila.

```
1 GROUP BY PRODUCTO.producto_id
2 ORDER BY cant_total DESC
3 LIMIT 1;
```

4.5.2. Vista de la factura

En este apartado se requiere mostrar la información necesaria para asemejarse a una factura de una orden.

Para esto creamos nuevamente la vista de la factura con el nombre de vista_factura pero al ser una factura y tener varios datos que seran consultados de diferentes tablas me concentre en ver esto como una vista separada; donde una sección sea el encabezado de la factura (que llevara folio, fecha, hora, total, rfc, nombre_cliente, su email, etc) y la otra sección sera como el detalle de la factura donde se incluiran los atributos relacionados al producto. Y al ser una factura se debe mostrar el subtotal a pagar de lo consumido en platillos o bebidas, la cual se hizo simplemente multiplicando la cantidad de los productos vendidos por su precio.

```
1 CREATE VIEW vista_factura AS
2 SELECT
3
4     ORDEN.folio,
5     ORDEN.fecha,
6     ORDEN.hora,
7     ORDEN.total,
8     CLIENTE_FACT.rfc,
9     CLIENTE_FACT.nombre_cliente,
10    CLIENTE_FACT.email,
11    CLIENTE_FACT.estado,
12    CLIENTE_FACT.codigo_postal,
13    CLIENTE_FACT.colonia,
14    CLIENTE_FACT.numero,
15    CLIENTE_FACT.calle,
16    CLIENTE_FACT.razon_social,
17
18    PRODUCTO.producto_id,
19    PRODUCTO.nombre,
20    PRODUCTO.precio,
21    DETALLE_ORDEN.cantidad,
22    (DETALLE_ORDEN.cantidad * PRODUCTO.precio) AS subtotal
```

Luego se usaron 3 JOINS para que a la hora de consultar la vista se visualice la información establecida anteriormente de las tablas ORDEN, PRODUCTO, CLIENTE_FACT y DETALLE_ORDEN, en una sola tabla. Uniendo primero ORDEN con CLIENTE_FACT por su rfc, luego ORDEN con DETALLE_ORDEN por su folio y una vez hecho esto poder unir el DETALLE_ORDEN con PRODUCTO gracias a producto_id, y con esto ya tendríamos la consulta con todos los atributos que nos interesen.

```
1 FROM ORDEN
2 JOIN CLIENTE_FACT ON ORDEN.rfc = CLIENTE_FACT.rfc
3 JOIN DETALLE_ORDEN ON ORDEN.folio = DETALLE_ORDEN.folio
4 JOIN PRODUCTO ON DETALLE_ORDEN.producto_id = PRODUCTO.producto_id
```

Y por último como una factura no se genera con sin un cliente, le especificamos con un WHERE a la base de datos que si hay un valor nulo en la parte de rfc filtre esos datos para no aparecer en nuestra factura.

```
1 WHERE ORDEN.rfc IS NOT NULL;
```

4.5.3. Vista productos no disponibles

Se hizo una vista para resolver el problema de obtener los nombres de los productos no disponibles.

Para esto se crea nuevamente la vista con el nombre vista_p_no_disponibles, como solo nos pide los nombres del producto será el unico atributo que mostraremos y simplemente con un WHERE le indicamos a la base que guiandose de los valores del atributo disponibilidad solo tome en cuenta aquellos que den FALSE.

```
1 CREATE VIEW vista\_p\_no\_disponibles AS
2 SELECT
3     PRODUCTO.nombre
4 FROM PRODUCTO
5 WHERE PRODUCTO.disponibilidad = FALSE;
```

4.5.4. Resultados Vistas

Se hicieron pruebas para cada una de las vistas generadas.

- Platillo más vendido: Para la vista de el platillo más vendido se insertaron 3 tacos de pastor con folio ORD-001 y 1 con folio ORD-002 y para un segundo platillo se inserto 0 Enchiladas verdes indicando que no se vendio ninguno. Nuestra salida

	producto_id integer	nombre character varying (40)	descripcion text	precio numeric (8,2)	cant_total bigint
1	1	Tacos de pastor	Tacos con carne al pas...	85.00	4

Figura 3: Resultado vista platillo más vendido

- Factura: Para la factura solo se deben mostrar ordenes con rfc, en esta prueba el folio ORD-001 tiene rfc entonces aparece y ORD-002 no tiene rfc entonces no aparece

	folio character varying (10)	nombre_cliente character varying (40)	razon_social character varying (150)	nombre character varying (40)	cantidad integer	subtotal numeric
1	ORD-001	Maria	Tacos El Pastor S.A. de C...	Tacos de pastor	3	255.00
2	ORD-001	Maria	Tacos El Pastor S.A. de C...	Agua de jamaica	2	50.00

Figura 4: Resultado Factura

- Productos no disponibles: Para esta prueba se uso el valor donde Enchiladas Verdes tiene disponibilidad = FALSE y Tacos y Agua de Jamaica tienen disponibilidad = TRUE. Entonces solo Enchilads Verdes aparecera.

	nombre character varying (40)
1	Enchiladas verdes

Figura 5: Resultado de producto no disponible

4.6. Índices

Para la creación de los índices primero analizamos que índices usar y en donde usarlos. Primero se usarán índices B-tree porque es el más versátil para operaciones de igualdad, ordenamiento y rango, lo cual para esta última será muy útil usar este tipo de índice además de que PostgreSQL ya tiene como predeterminado estos índices.

Estos índices se aplicarán a llaves foráneas donde los atributos no sean una clave principal, esto se hace para acelerar los cruces de datos y operaciones de búsqueda. Entonces tenemos como FK's, que no tengan primary key o unique:

- detalle_orden.producto_id
- orden.rfc
- orden.num_empleado

Con esto podemos comenzar a crear nuestros índices

```

1  -- Para producto_id
2  CREATE INDEX idx_DetalleOrden_producto_id ON DETALLE_ORDEN(producto_id);
3
4  -- Para rfc
5  CREATE INDEX idx_Orden_rfc ON ORDEN(rfc);
6
7  -- Para num_empleado
8  CREATE INDEX idx_Orden_num_empleado ON ORDEN(num_empleado);

```

Y por último hacemos un índice para la 'fecha' ya que para la parte donde estén las funciones de consulta en la función de ventas por rango de fechas, se está usando un rango. Donde el B-tree es ideal para rangos porque no desordena los valores y sigue una secuencia en su escaneo del límite inferior hasta el superior sin recorrer toda la tabla.

```

1  CREATE INDEX idx_orden_fecha ON ORDEN (fecha);

```

5. Presentación

5.1. Fase de exportación de la BD origen a txt

Conectamos la base origen con la base destino, como se puede ver en la siguiente imagen.

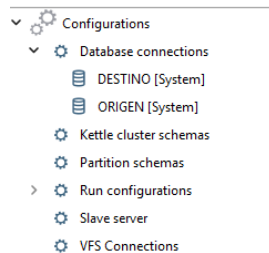


Figura 6: Menú para realizar la conexión de las bases.

Nos saldrá un cuadro de menú que servirá para confirmar la conexión de nuestras bases de datos.

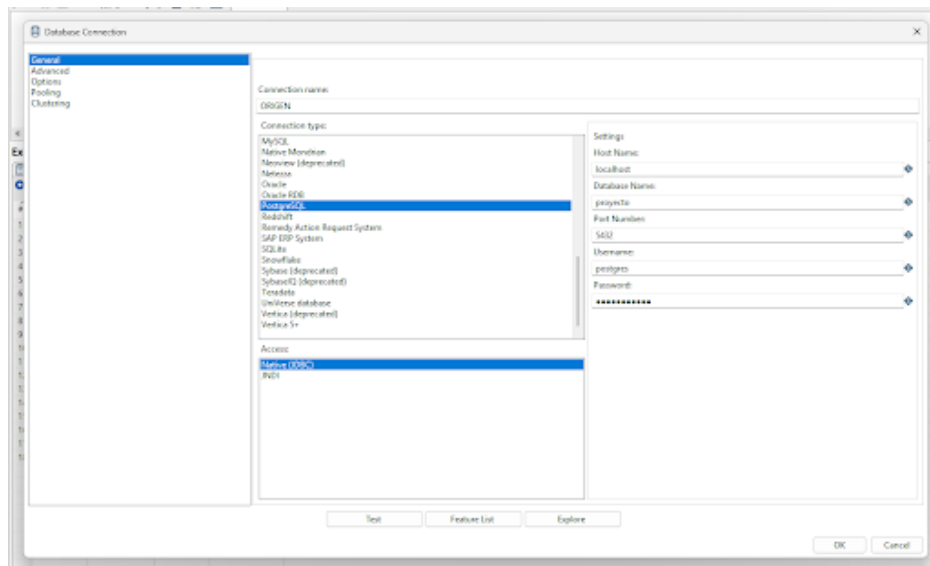


Figura 7: Confirmación detallada para la conexión de las bases.

Para que al final nos mande un mensaje de que se logró la conexión entre las bases.

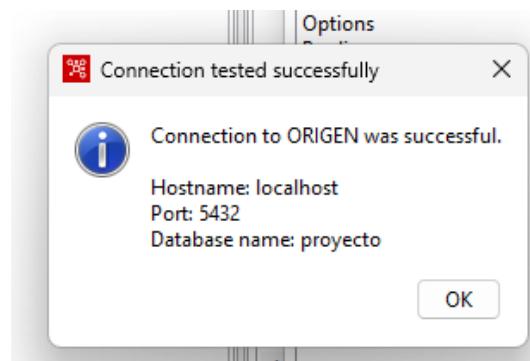


Figura 8: Confirmación de que se conectaron las bases.

Tras establecer la conexión entre ambas bases de datos, se procede con la fase de extracción. Para ello, se diseñaron transformaciones individuales en Pentaho Data Integration encargadas de migrar los registros de las tablas más relevantes de PostgreSQL hacia archivos de texto plano (.txt). Con el fin de mantener un orden estructurado, se adoptó la nomenclatura estándar **TRANS_export_nombreTabla**.

En este caso se analizará cómo pasar nuestra tabla orden a un archivo (.txt). Primero conectaremos el bloque llamado **Leer orden** a nuestra base de origen, donde se le pedirá a la base de datos que nos dé el folio, fecha, hora, total, num_empleado y rfc, y que ordene los datos por su folio.

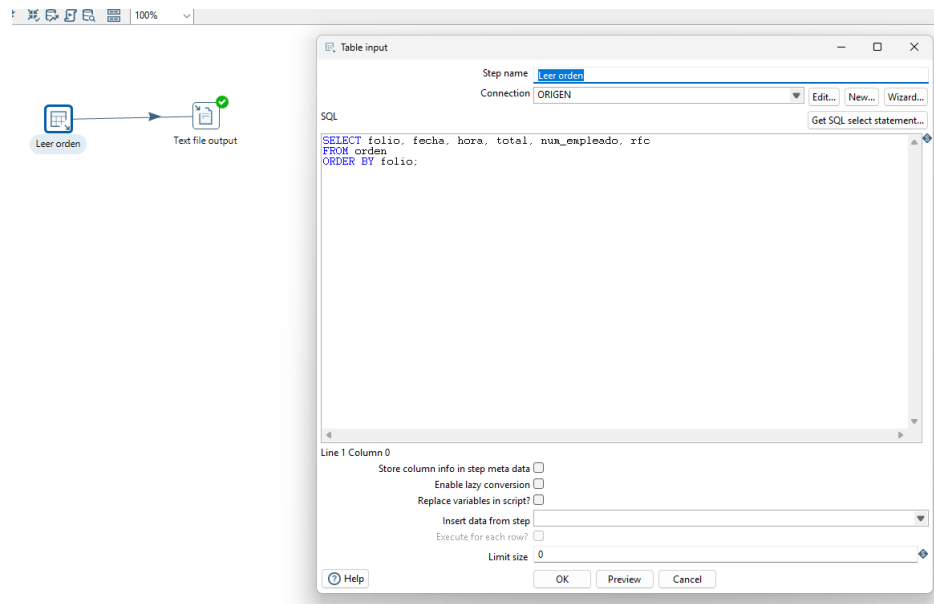


Figura 9: Table Input

Después le indicamos dónde queremos guardar el archivo (.txt) que se generará una vez terminado el proceso y le damos **Save**.

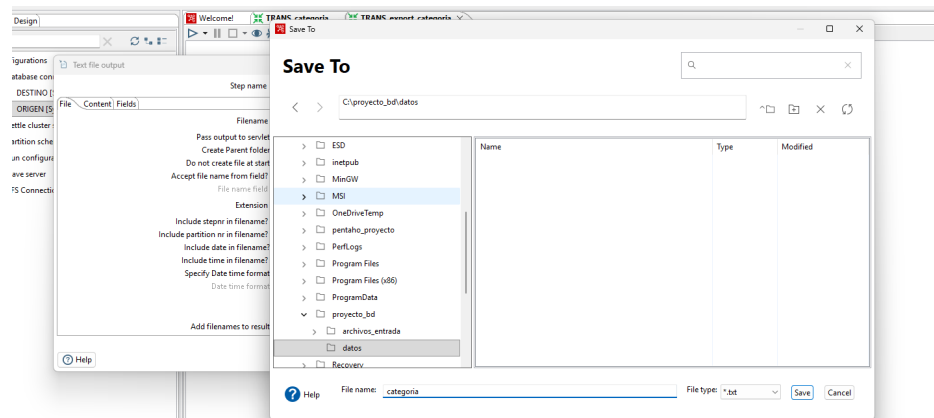


Figura 10: Elección de dónde se va a guardar.

Una vez que le damos **Save**, tendremos que configurar el **Text file output** para crear el archivo (.txt), donde tendremos que indicar dónde está ubicado. En la pestaña de **Content** se tendrá que seleccionar el cuadrado de **Header** para que, a la hora de crear el archivo (.txt), se separe con líneas verticales '|' e indique qué significa cada dato con su atributo establecido.

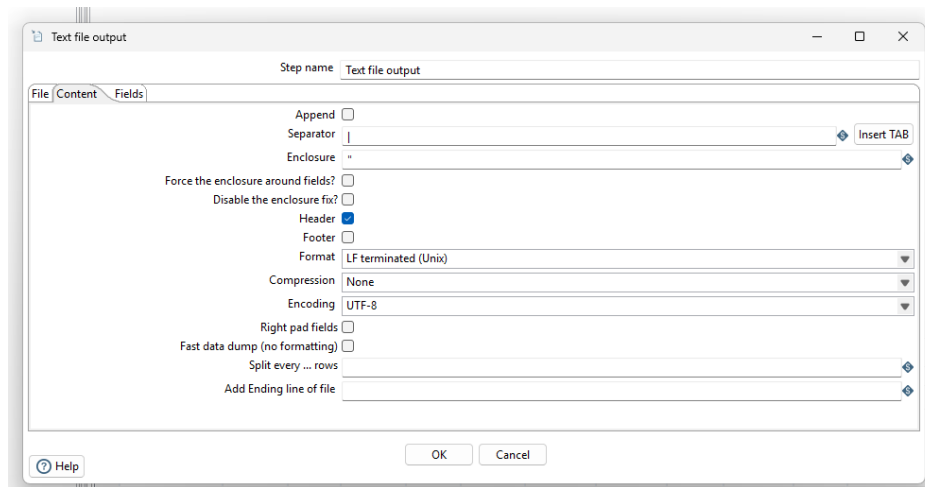


Figura 11: Configuración antes de generar el archivo (.txt).

Una vez hecho esto, nos vamos a la sección de **Fields** y le damos a la función **Get Fields**, disponible en la interfaz del componente. Esta función automatiza la lectura de los datos procedentes de la consulta **SQL** previa, importando de forma automática los nombres de los atributos junto con sus respectivos tipos de datos y longitudes.

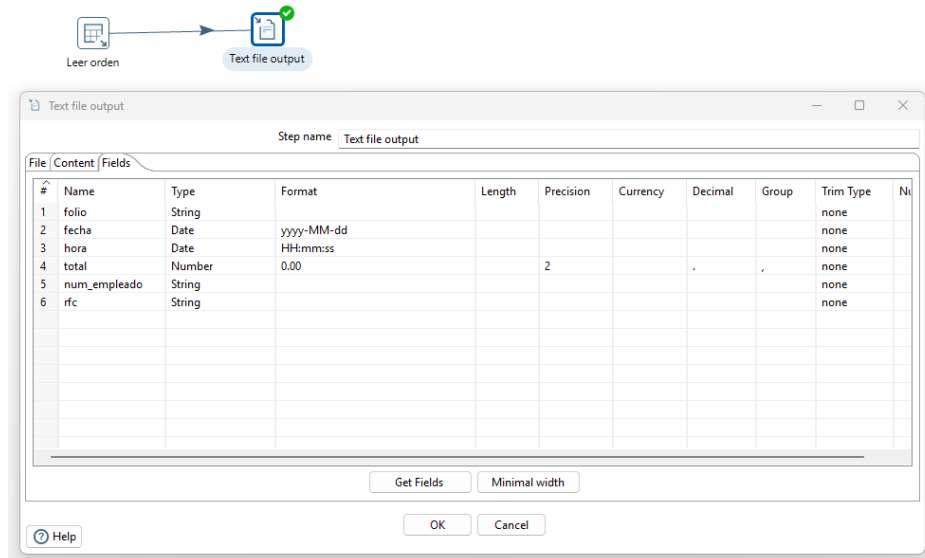
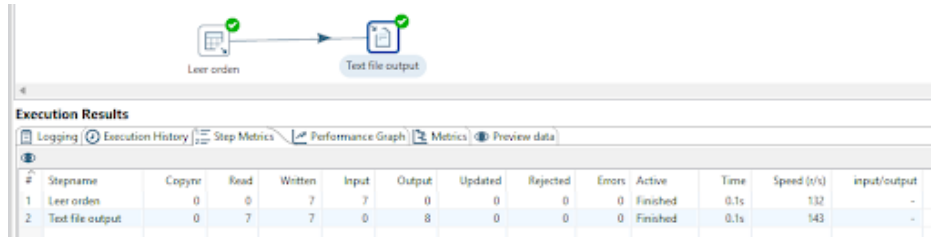


Figura 12: Resultado de la función Get Fields.

Ya que tengamos configurado lo necesario para la transformación, se procedió con la ejecución del flujo de datos, donde:

- Se inicia el procesamiento mediante la opción **Run**, lo que inicializa el motor de **Pentaho**.
- Luego se verifica con **Execution Results** que nuestra migración de datos fue realizada con éxito. Para esto, debemos ver en la tabla **Metrics** y verificar que en **Stepname ->Leer orden** se encuentre **Read = 7**. También se debe verificar que en **Stepname ->Text file Output** se encuentre **Written = 7**.



Execution Results

#	Stepname	Copied	Read	Written	Input	Output	Updated	Rejected	Errors	Active	Time	Speed (s/s)	input/output
1	Leer orden	0	0	7	7	0	0	0	0	Finished	0.1s	132	-
2	Text file output	0	7	7	0	8	0	0	0	Finished	0.1s	143	-

Figura 13: Tabla de Metrics.

Finalmente, corroboramos que sí se hizo el archivo (.txt) en la dirección de memoria que le habíamos indicado. Una vez ubicado, se constató que el archivo realizó de buena manera la estructuración de nuestros datos conforme a las restricciones que le habíamos dado anteriormente.

```

Archivo  Editar  Ver
folio|fecha|hora|total|num_empleado|rfc
ORD-001|2026-05-29|21:12:45|195.00|EMP-001|
ORD-002|2026-05-29|21:12:45|275.00|EMP-002|
ORD-003|2026-05-29|21:12:45|280.00|EMP-006|
ORD-004|2026-05-29|21:12:45|150.00|EMP-001|GOPE800101AAA
ORD-005|2026-05-29|21:12:45|415.00|EMP-002|MARM750615BBB
ORD-006|2026-05-29|21:12:45|290.00|EMP-010|
ORD-007|2026-05-29|21:12:45|190.00|EMP-010|ROPE850101DDD

```

Figura 14: Correcta estructuración de nuestro archivo (.txt).

Una vez hecho esto, se generó el archivo (.ktr). Este tipo de archivo representa los artefactos de software generados por la herramienta *Pentaho*. A nivel estructural, corresponden a archivos en formato XML que contienen los datos y la lógica de un flujo. Esto nos sirve en nuestro proyecto, ya que de cierta forma automatiza los procesos, lo que quiere decir que cada vez que quisiéramos generar `orden.txt`, simplemente sería abrir el archivo (.ktr), correrlo y no tener que volver a hacer todo el proceso.



Figura 15: Archivo (.ktr) generado.

5.2. Fase de importación de txt a la BD destino

Una vez generados los archivos .txt con los datos de origen, el siguiente paso fue cargarlos en la base de datos de destino, para esto, creamos nuevas transformaciones en *Pentaho Data Integration*, aplicando la nomenclatura `TRANS_import_nombreTabla`.

En estas transformaciones, el proceso se invierte, como primer paso, se utilizó la herramienta `Text file input` para leer el archivo de texto generado anteriormente.

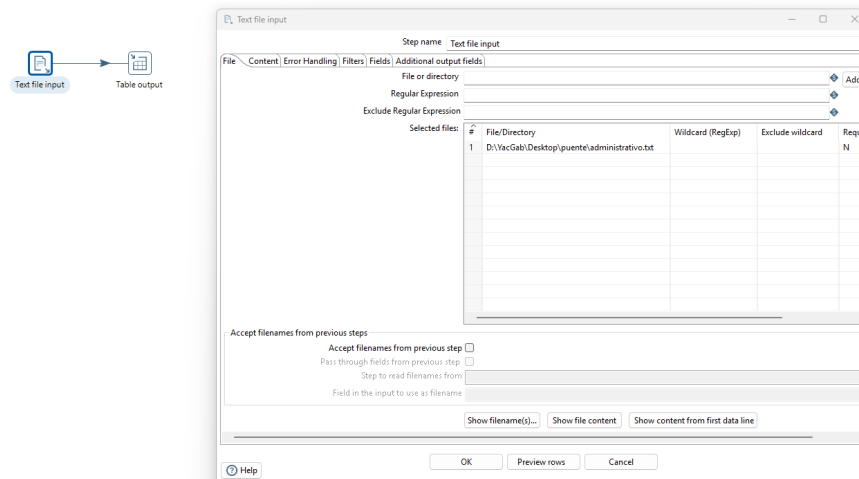


Figura 16: Estructura básica de la transformación de importación.

Para configurar el componente de lectura, en la pestaña **File**, seleccionamos el archivo correspondiente (por ejemplo, **administrativo.txt**) y usamos el botón **Add** para agregarlo a la lista de archivos seleccionados.

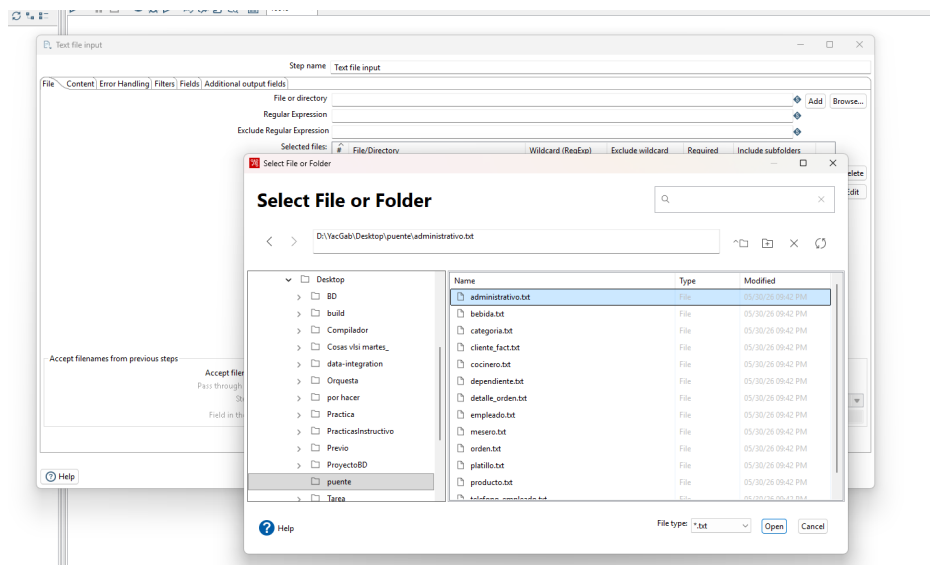


Figura 17: Selección del archivo de texto a importar.

Posteriormente, en la pestaña **Content**, indicamos que nuestro archivo está separado por barras verticales (|) y que cuenta con una fila de encabezado.

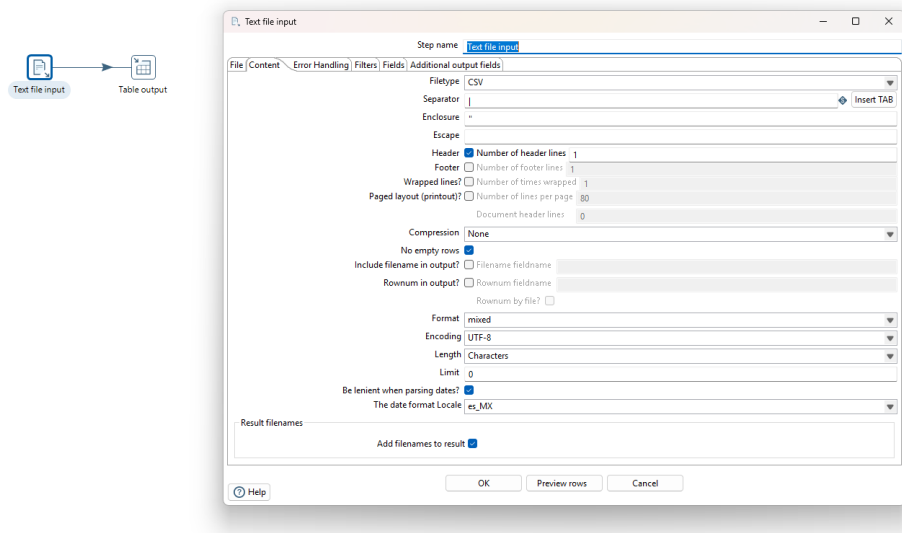


Figura 18: Configuración del separador y contenido del texto.

Finalmente, en la pestaña **Fields**, utilizamos el botón **Get Fields** para que la herramienta detectara automáticamente las columnas del archivo de texto y sus tipos de datos.

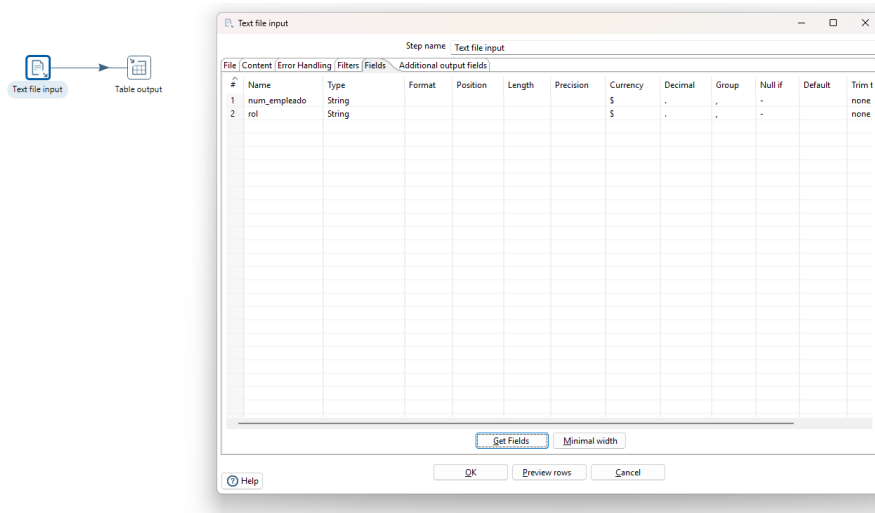


Figura 19: Detección automática de los campos a importar.

El segundo paso de la transformación consistió en configurar el componente **Table output**, el cual se encarga de insertar los datos en PostgreSQL. Aquí configuramos la conexión hacia la base de datos Destino, especificamos la tabla a llenar (por ejemplo, **administrativo**) y marcamos la opción **Specify database fields** para mapear las columnas correctamente.

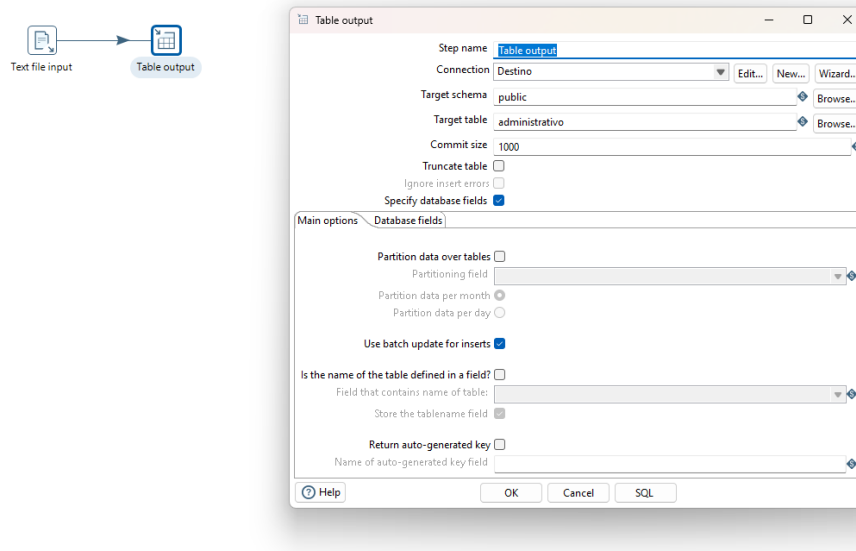


Figura 20: Configuración principal de la salida a base de datos.

Para facilitar el proceso y evitar errores de dedo al escribir el nombre de la tabla, aprovechamos el explorador de bases de datos que incluye *Pentaho*, esta ventanita nos permite buscar y seleccionar nuestra tabla destino de forma visual y directa (en este caso, la tabla **administrativo**).

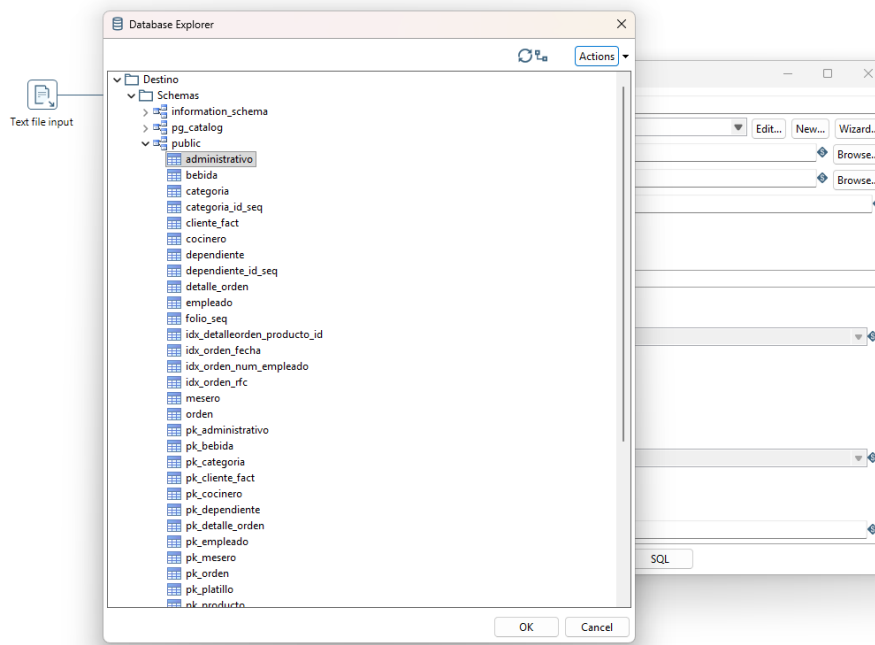


Figura 21: Selección visual de la tabla destino mediante el explorador de bases de datos.

Una vez seleccionada la tabla, nos pasamos a la pestaña **Database fields**, aquí nos encargamos de emparejar los datos que vienen del archivo **.txt** con las columnas exactas que tenemos en nuestra tabla de la base de datos, asegurando que cada dato caiga exactamente en su lugar.

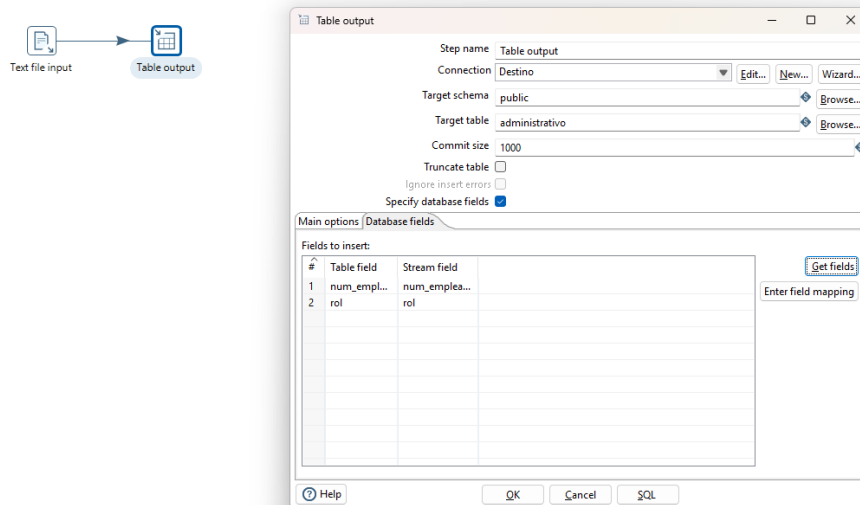


Figura 22: Emparejamiento de los campos entrantes hacia las columnas de la base de datos destino.

5.3. Orquestación y automatización (Trabajos/Jobs)

Al tener las 13 transformaciones para exportar y las 13 transformaciones para importar listas, el proceso manual de ejecutarlas una por una resultaba ineficiente, por esta razón se diseñaron **Trabajos (Jobs)** en *Pentaho* con extensión **.kjb**, los cuales funcionan como orquestadores para ejecutar las transformaciones en cadena.

Primero, creamos un flujo para el proceso de extracción (Conversor), este trabajo ejecuta todas las transformaciones de exportación en secuencia, además, se implementó un manejo de errores; si alguna tabla falla al exportarse, el flujo se desvía mediante una flecha roja hacia un archivo de **Log (Registro)**, permitiendo guardar los detalles del error sin detener el sistema por completo.



Figura 23: Job encargado de orquestar la exportación de la base de datos a archivos de texto.

De igual manera, creamos un Job final para la fase de carga (Importador), este trabajo está diseñado respetando el orden estricto de las llaves foráneas (cargando primero los catálogos y al final las tablas transaccionales como las órdenes), adicionalmente, se agregaron **Scripts SQL** al inicio y al final del flujo para preparar la base de datos (por ejemplo, deshabilitando restricciones temporales o limpiando tablas) asegurando así que, con un solo clic, toda la información se migre de forma limpia, automática y sin errores de duplicidad.

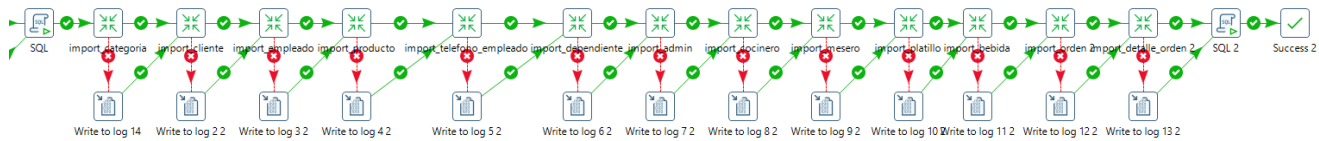


Figura 24: Job maestro encargado de la importación final y manejo de errores.

6. Conclusiones

- Bojorquez Covarrubias Evans Martin:

Al programar las funciones de consulta y armar mis propios scripts de prueba, prevenir valores nulos usando COALESCE, o evitar que todo el sistema colapsara ante un dato incorrecto usando excepciones, me hizo entender que en un futuro en el ámbito laboral los sistemas van a recibir errores constantemente y tendré que saber controlarlos, además, tenía que integrar mi código con el trabajo de mi equipo por lo mismo tenía que tener un código ordenado y bien documentado; gracias a la realización de este proyecto, me doy cuenta que la base de datos de cualquier negocio es casi el pilar de todo y por lo mismo debe ser diseñada para ser segura, estable y a prueba de errores.

- Gabriel Hernández Yukioayax Canek:

Durante el desarrollo de este proyecto comprobé que tener una buena estructura desde el inicio lo es todo, al trabajar en el diseño de las tablas y definir cómo se relaciona cada elemento del restaurante, entendí que configurar correctamente las llaves primarias y foráneas te salva de muchos dolores de cabeza y evita que el sistema se llene de datos basura, además, ver cómo funcionan los triggers en la práctica fue muy interesante, porque te das cuenta de que la base de datos puede trabajar en automático (como al calcular los totales o generar los folios) sin que el usuario tenga que hacer cálculos a mano, me llevo el gran aprendizaje de que hacer las cosas con orden y lógica desde el diseño inicial es lo que hace que un sistema realmente funcione en la vida real.

- Martínez Ortiz Eydan:

En el desarrollo del proyecto puede entender de mejor manera como funcionan las vistas, aprendí que es una manera más sencilla de llamar consultas ya que sirve como un atajo para consultar cosas más específicas de las tablas, se me hizo como un SELECT pero mejorado. Hubo problemas en la parte de la vista para la factura ya que al usar varias entidades de las tablas tenía que usar algunos más JOINS de los que necesitaba aunque no se si esa era la mejor solución, por lo demás no hubo tantos problemas los indices no fueron tanto problema solo tuve que consultar en que casos eran mejor usarlos y así determinar en donde poner los indices. También la parte de la orquestación fue tediosa el hecho de entender como pasar de la base de datos de origen a la de destino, ahí es donde conocí los archivos (.ktr) y entendí que con esos se automatizaban los procesos cosa que hacía más fácil el trabajo. Aparte de eso el proyecto fue algo tedioso y más con la parte de orquestación aunque en conjunto entre todo el equipo se logro hacer.

- Parra Bello Carlos Enrique:

Este proyecto me ayudó a entender mejor cómo se construye una base de datos completa desde cero y cómo cada componente tiene una función importante dentro del sistema. La parte que más me gustó fue trabajar con las funciones y los triggers, ya que permiten automatizar procesos y evitar errores que podrían ocurrir si todo se hiciera manualmente. También pude reforzar conceptos sobre relaciones entre tablas, integridad de los datos y documentación de objetos de base de datos. A lo largo del desarrollo me di cuenta de que una buena planeación desde el diseño facilita mucho el trabajo posterior y hace que el sistema sea más fácil de mantener. En general, fue una experiencia

que me permitió poner en práctica varios temas del curso y comprender mejor cómo se aplican en un proyecto real.

- Parra Vera Héctor Jesús

Llevar el proyecto del modelo teórico a una base de datos funcional fue lo más interesante del curso. Pasamos del diseño en papel a ver cómo el modelo relacional, sustentado en álgebra relacional, sostiene un sistema real. Y aunque nuestro restaurante es ficticio, entendimos que industrias enteras dependen de estos mismos principios.

Uno de los mayores retos fue la programación de los triggers, sobre todo entender en qué momento debían dispararse. Al inicio asumíamos que todos debían ejecutarse antes de la operación (**BEFORE**), hasta que con el de eliminación nos topamos con que el total solo se podía recalcular bien una vez que la fila ya había sido borrada; por eso terminó siendo **AFTER DELETE**. Lograr que el total de la orden se mantuviera consistente al insertar, modificar y eliminar productos fue uno de los aciertos que más nos dejó satisfechos.

- Yañez Barajas Brandon

Mi parte del proyecto fue enfocado más la orquestación de datos con Pentaho, pasando la información de la base operativa a la de destino usando archivos de texto como punto intermedio. Algo que me quedó muy claro es que si las tablas no están bien definidas desde el inicio, la orquestación se vuelve un problema detrás de otro; por ejemplo, el orden de carga lo dictan las llaves foráneas y si no tienes claro qué tabla depende de cuál, nada entra. También me enfrenté a detalles técnicos que no esperaba, como que los campos **TEXT** de PostgreSQL llegaban a Pentaho con longitudes enormes que tronaban la escritura de los archivos, o que el tipo **TIME** internamente se maneja como **Date** y hay que controlar el formato para que salga correctamente. La parte de las validaciones me hizo entender que no se trata solo de mover datos de un lugar a otro, sino de asegurarse de que lo que entra esté bien y que si algo falla, el flujo no se caiga completo. Al final me llevo que un buen diseño de base de datos no solo facilita las consultas, también hace posible que procesos como la migración funcionen sin sorpresas.